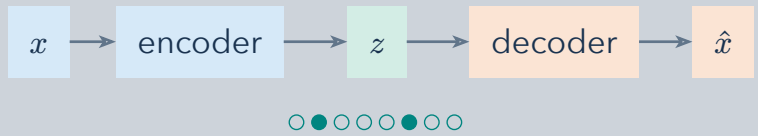
Every autoencoder learns to reconstruct an input through an intermediate representation. Compare the changing constraint or architecture while the common data path stays fixed.

1 Autoencoder

compact bottleneck

Encode x into a bottleneck z, then reconstruct $\hat{x}$. A reconstruction loss rewards retaining useful information.

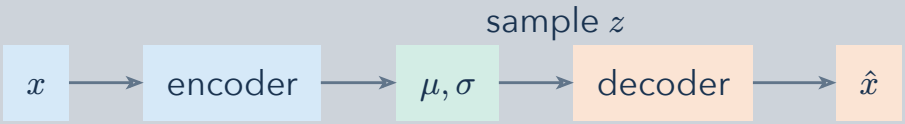
2 Sparse autoencoder



few active components

Penalize latent activity so only a few components activate for one input. The latent layer may be wider than the input; sparsity is the constraint.

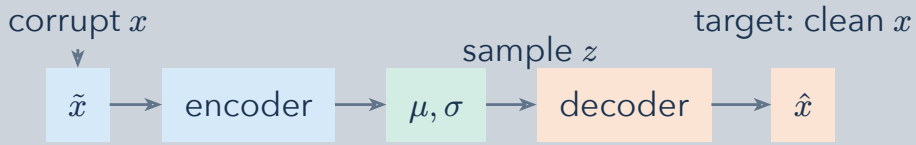
3 Variational autoencoder



$$z = \mu + \sigma \cdot \varepsilon$$
$$\varepsilon \sim \mathcal{N}(0, I)$$

The encoder predicts a distribution over z, not just one code. Reconstruction and a KL-divergence term train it toward an explicit prior.

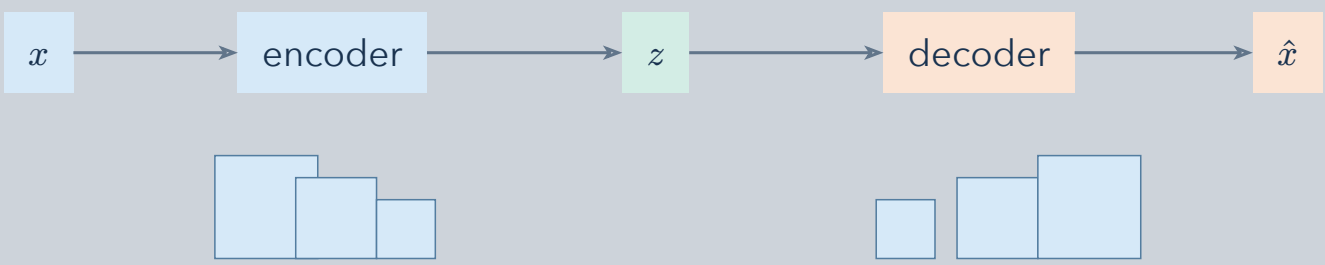
4 Denoising variational autoencoder



$$z = \mu + \sigma \cdot \varepsilon$$
$$\varepsilon \sim \mathcal{N}(0, I)$$

Corrupt the input, then reconstruct the clean target. The stochastic latent step remains; robustness and distribution learning are separate ingredients.

5 Convolutional autoencoder



spatial feature maps

Convolutions share filters over spatial locations. Feature maps change resolution through the encoder and decoder; this is an architectural choice.

6 These choices can be combined

representation

bottleneck · sparse · stochastic

architecture and training

convolutional · denoising

A convolutional model can also be sparse, variational, or denoising. These names describe different design dimensions, not mutually exclusive model families.

Notation: x = input, $\hat{x}$ = reconstruction, z = latent code, μ and σ = latent mean and standard deviation. The sampled noise ε enables the reparameterization step.